

Cracking the Code: Mastering Pseudocode for OOP Interviews

Welcome, aspiring software engineers! This presentation delves into a series of common pseudocode questions frequently encountered in technical interviews at leading companies like Infosys, Capgemini, Cognizant ,Genpact ,TCS , Wipro ,FICO, HCL and Accenture. We'll explore core programming concepts, analyze pseudocode, and determine the correct outputs. Let's sharpen your problem-solving skills together.

NK by Neeraj Khanna

Q1. What will be the output of the following pseudocode?

```
class Animal {
    void sound() { print "Animal"; }
}
class Dog extends Animal {
    void sound() { print "Dog"; }
}
Animal a = new Dog();
a.sound();
```

A) Animal

B) Dog

C) Animal Dog

D) Error

Explanation: Polymorphism

This question tests your understanding of polymorphism and method overriding. When `Animal a = new Dog();` is executed, the object 'a' is instantiated as a `Dog`, even though its reference type is `Animal`.

The `a.sound();` call demonstrates dynamic dispatch; the JVM determines the actual method to call at runtime based on the object's true type (`Dog`), not the reference type (`Animal`). Thus, the `sound()` method of the `Dog` class is invoked, printing "Dog".

Correct Answer: B) Dog

Q2. What will be the output of the following pseudocode?

```
class Parent {
    Parent() { print "Parent"; }
}
class Child extends Parent {
    Child() { print "Child"; }
}
Child c = new Child();
```

A) Parent

B) Child

C) Parent Child

D) Child Parent

Explanation: Constructor Chaining

This question assesses knowledge of constructor invocation in inheritance. In object-oriented languages, when a child class object is created, the parent class's default (no-argument) constructor is implicitly called before the child class's constructor.

Therefore, when `Child c = new Child();` is executed, the `Parent()` constructor runs first, printing "Parent", followed by the `Child()` constructor, which prints "Child".

Correct Answer: C) Parent Child

Q3. What will be the output of the following pseudocode?

```
class A {
    static void show() { print "A"; }
}
class B extends A {
    static void show() { print "B"; }
}
A obj = new B();
obj.show();
```

A) A

B) B

C) Error

D) AB

Explanation: Static Method Hiding

This question delves into static methods and inheritance. Unlike instance methods, static methods are not subject to polymorphism or overriding. When a static method with the same signature is defined in a child class, it's called "method hiding," not overriding.

The method invoked depends on the reference type, not the actual object type. Since `obj` is declared as type `A`, `A.show()` is called at compile time, printing "A".

Correct Answer: A) A

Q4. What will be the output of the following pseudocode?

```
abstract class Shape {
    abstract void draw();
}
Shape s = new Shape();
```

A) Compiles successfully

B) Error: Cannot instantiate abstract class

C) Runtime error

D) No output

Explanation: Abstract Class Instantiation

This question tests your knowledge of abstract classes. An abstract class is a class that cannot be instantiated on its own; it must be subclassed, and its abstract methods must be implemented by a concrete subclass.

The line `Shape s = new Shape();` attempts to create an object directly from the abstract class `Shape`, which is forbidden. This will result in a compile-time error.

Correct Answer: B) Error: Cannot instantiate abstract class

Q5. What will be the output of the following pseudocode?

```
interface A { void show(); }
class B implements A {
    public void show() { print "B"; }
}
A obj = new B();
obj.show();
```

A) A

B) B

C) Error

D) AB

Explanation: Interface Implementation

This question examines interfaces and their implementation. An interface defines a contract that classes can implement. When class `B` implements interface `A`, it must provide an implementation for all methods declared in `A`.

Similar to polymorphism with classes, when `A obj = new B();` is executed, `obj` refers to an instance of `B`. Calling `obj.show();` invokes the implementation of `show()` provided by class `B`, which prints "B".

Correct Answer: B) B

Q6. What will be the output of the following pseudocode?

```
class Test {
    void fun(int x) { print "int"; }
    void fun(double x) { print "double"; }
}
Test t = new Test();
t.fun(5.0);
```

A) int

B) double

C) Error

D) int double

Explanation: Method Overloading

This question tests your understanding of method overloading. Method overloading allows a class to have multiple methods with the same name but different parameter lists. The compiler determines which method to call based on the number and types of arguments passed during the method invocation.

When `t.fun(5.0);` is called, the argument `5.0` is a double literal. The compiler finds the `fun(double x)` method as the most specific match and invokes it, printing "double".

Correct Answer: B) double

Q7. What will be the output of the following pseudocode?

```
class Person {
    private int age;
    public void setAge(int a) { age = a; }
    public int getAge() { return age; }
}
Person p = new Person();
p.setAge(25);
print p.getAge();
```

A) 0

B) Null

C) Error

D) 25

Explanation: Encapsulation (Getters and Setters)

This question focuses on encapsulation, a core OOP principle. Encapsulation involves bundling data (attributes) and methods (functions) that operate on the data within a single unit (class). It also restricts direct access to some of the object's components, often by using access modifiers like `private`.

Here, `age` is a `private` variable, accessed and modified via `public` `setAge()` and `getAge()` methods. After `p.setAge(25);`, the `age` variable within the `p` object becomes 25. Subsequently, `p.getAge();` retrieves this stored value, printing "25".

Correct Answer: D) 25

Q8. What will be the output of the following pseudocode?

```
class Parent {
    final void show() { print "Parent"; }
}
class Child extends Parent {
    void show() { print "Child"; }
}
Child c = new Child();
c.show();
```

A) Parent

B) Child

C) Error: Cannot override final method

D) No output

Explanation: Final Methods

This question tests your understanding of the `final` keyword in OOP. When a method is declared as `final`, it cannot be overridden by any subclass. This mechanism is used to prevent unintended modification of behavior in an inheritance hierarchy and ensures that the method's implementation remains consistent.

Attempting to override a `final` method, as seen in the `Child` class, will result in a compile-time error.

Correct Answer: C) Error: Cannot override final method

Q9. What will be the output of the following pseudocode?

```
class A { }
class B extends A { }
A a = new B();
B b = (B)a;
print "Success";
```

A) Success

B) Error

C) Runtime Exception

D) Compilation Error

Explanation: Type Casting

This question tests your understanding of type casting in an inheritance hierarchy.

- `A a = new B();`: This is upcasting, where an object of a subclass (`B`) is assigned to a reference of its superclass (`A`). This is always safe and allowed.
- `B b = (B)a;`: This is downcasting, where a superclass reference (`a`) is explicitly cast to a subclass type (`B`). This is only safe and allowed if the object actually referred to by the superclass reference is, in fact, an instance of the subclass (or a subclass thereof). In this case, `a` truly holds a `B` object.

Since the object referenced by `a` is indeed a `B`, the downcast is successful, and "Success" is printed.

Correct Answer: A) Success

Q10. What will be the output of the following pseudocode?

```
interface A { void show(); }
interface B { void display(); }
class C implements A, B {
        public void display() { print "B"; }
    public void show() { print "A"; }
}
C obj = new C();
obj.show();
obj.display();
```

A) A B

B) B A

C) Error

D) A B

Explanation: Multiple Interface Implementation

This question tests your understanding of multiple inheritance through interface hierarchy.

This pseudocode demonstrates a class C implementing two interfaces, A and B. Both interface methods, show() and display(), are correctly implemented within class C. When an object obj of class C is created, calling obj.show() will execute the show() method, printing "A". Subsequently, calling obj.display() will execute the display() method, printing "B". The output will be "A" followed by "B".

Correct Answer: D) A B

Q11. What will be the output of the following pseudocode?

```
class Demo {
    static { print "Static"; }
    Demo() { print "Constructor"; }
}
Demo d = new Demo();
```

A) Constructor Static

B) Constructor

C) Static Constructor

D) Static

Explanation: Analyzing Static Block and Constructor Execution order

In many programming languages (like Java, from which this pseudocode derives its logic), static blocks are executed when the class is loaded into memory, which happens **before** any objects of that class are created or any static members are accessed. Constructors, on the other hand, are executed when an object of the class is instantiated. Therefore, when **Demo d = new Demo();** is executed, the static block will run first, printing "Static", and then the constructor will run, printing "Constructor".

Correct Answer: C) Static Constructor

Q12. What will be the output of the following pseudocode?

```
class Parent {
    int x = 10;
}
class Child extends Parent {
    int x = 20;
}
Child c = new Child();
print c.x;
```

A) 10

B) 20

C) Error

D) 0

Explanation: Variable Shadowing in inheritance

This scenario illustrates variable shadowing (or hiding) in inheritance. Both the **Parent** class and the **Child** class declare a variable named **x**. When an instance of the **Child** class is created, and **c.x** is accessed directly, the variable **x** declared within the **Child** class takes precedence.

The **Child**'s **x** (with value 20) hides the **Parent**'s **x** (with value 10) for direct access through the **Child** object.

Correct Answer: B) 20

Q13. What will be the output of the following pseudocode?

```
abstract class Vehicle {
    abstract void run();
}
class Bike extends Vehicle {
    void run() { print "Bike running"; }
}
Vehicle v = new Bike();
v.run();
```

A) Vehicle running

B) Bike running

C) Error

D) No output

Explanation: Polymorphism with abstract Methods and Inheritance

This pseudocode demonstrates polymorphism using an abstract class and method overriding. The **Vehicle** class is abstract and declares an abstract method **run()**. The **Bike** class extends **Vehicle** and provides a concrete implementation for the **run()** method.

When **Vehicle v = new Bike();** is executed, a **Bike** object is created and assigned to a **Vehicle** reference. Despite the reference type being **Vehicle**, the actual object type is **Bike**. Therefore, calling **v.run()** will invoke the overridden **run()** method in the **Bike** class.

Correct Answer: B) Bike running

Q14. What will be the output of the following pseudocode?

```
class Test {
    Test() { print "No arg"; }
    Test(int x) { print "Int arg"; }
}
Test t = new Test(5);
```

A) No arg

B) Error

C) No arg Int arg

D) Int arg

Explanation: Constructor Invocation with Arguments

This example showcases constructor overloading. The **Test** class has two constructors: a default (no-argument) constructor and a parameterized constructor that takes an integer argument.

The line **Test t = new Test(5);** explicitly calls the parameterized constructor, passing the integer value 5. Therefore, only the code inside the **Test(int x)** constructor will be executed, printing "Int arg". The no-argument constructor is not invoked.

Correct Answer: D) Int arg

Q15. What will be the output of the following pseudocode?

```
class A {
    int x = 5;
}
A a1 = new A();
A a2 = a1;
a2.x = 10;
print a1.x;
```

A) 5

B) 10

C) Error

D) Null

Explanation: Object References and Data Manipulation

This pseudocode demonstrates how object references work. Initially, **a1** refers to a new object of class **A** with its **x** value set to 5.

The line **A a2 = a1;** does not create a new object; instead, it makes **a2** refer to the **same** object that **a1** refers to. Both **a1** and **a2** are now pointing to the exact same memory location.

When **a2.x = 10;** is executed, the **x** value of the shared object is modified to 10. Consequently, when **a1.x** is printed, it will reflect this change.

Correct Answer: B) 10

Q16. What will be the output of the following pseudocode?

```
class Parent {
    void show() { print "Parent"; }
}
class Child extends Parent {
    void show() {
        super.show();
        print "Child";
    }
}
Child c = new Child();
c.show();
```

A) Parent

B) Child

C) Parent Child

D) Child Parent

Explanation: Method overriding and Calling Parent Method

This pseudocode demonstrates method overriding and the use of the **super** keyword. The **Child** class overrides the **show()** method from its **Parent** class.

Inside the **Child**'s **show()** method, **super.show();** explicitly calls the **show()** method of the **Parent** class first, which prints "Parent". After that, the line **print "Child";** is executed.

Therefore, when **c.show()** is called on a **Child** object, both outputs will appear sequentially.

Correct Answer: C) Parent Child

Q17. What will be the output of the following pseudocode?

```
interface A {
    default void show() { print "A"; }
}
class B implements A { }
B b = new B();
b.show();
```

A) A
B) Error
C) No output
D) Null

Explanation: Invoking Default Methods in Interface Implementations

This pseudocode demonstrates a feature present in modern programming languages (like Java 8+): default methods in interfaces. A default method provides a concrete implementation within an interface. Classes implementing this interface automatically inherit and can use this default implementation without needing to override it.

Since class **B** implements interface **A** and does not provide its own implementation for **show()**, calling **b.show()** will invoke the default **show()** method from interface **A**.

Correct Answer: A) A

Q18. What will be the output of the following pseudocode?

```
final class A
{
}
class B extends A
{
}
B b = new B();
```

A) Error
B) Success
C) No output
D) Compilation warning

Explanation: Immutability of Final Classes in Inheritance

In many object-oriented programming languages, the **final** keyword (or equivalent) when applied to a class, means that the class cannot be subclassed or inherited from. This is often used for security reasons, to create immutable classes, or to prevent unintended extensions.

Here, **class A** is declared as **final**. Attempting to extend a **final** class, as **class B extends A** does, will result in a compilation error. The pseudocode would not successfully compile or run.

Correct Answer: A) Error

Q19. What will be the output of the following pseudocode?

```
class A { }
class B extends A { }
A a = new A();
B b = (B)a;
print "Done";
```

A) Done
B) Error
C) Runtime Exception
D) Compilation Error

Explanation: Type Casting and Runtime Errors

This pseudocode attempts to cast an instance of class **A** to class **B**. While **B** extends **A**, an instance of **A** is not an instance of **B**. This downcasting operation will result in a **Runtime Exception** (specifically, a ClassCastException in Java or similar in other languages) because the object 'a' does not represent an object of type 'B'.

Correct Answer: C) Runtime Exception

Q20. What will be the output of the following pseudocode?

```
class A {
    private void show() { print "A"; }
}
class B extends A {
    void show() { print "B"; }
}
B b = new B();
b.show();
```

A) A
B) B
C) Error
D) No output

Explanation: Method Overriding and Private Method

In this scenario, the **show()** method in class **A** is **private**. Private methods are not inherited by subclasses. Therefore, the **show()** method in class **B** is an entirely new method, not an override. When **b.show()** is called, the **show()** method of class **B** is executed.

Correct Answer: B) B

Q21. What will be the output of the following pseudocode?

Company: Wipro , Capgemini

```
class A {
    A(int a) { print "A"; }
}
class B extends A {
    B() { print "B"; }
}
B b = new B();
```

A) A B

B) B A

C) Error

D) AB

Explanation: Constructor Chaining in Inheritance

When an object of a subclass is created, the constructor of the superclass is implicitly called first, before the subclass's own constructor. This is known as constructor chaining. Here the super class contains only the parameterized constructor. The compiler will only provide the super() call to invoke only the default constructor(Non parametrized constructor) so the program will generate an error

Correct Answer: C)Error

Q22. What will be the output of the following pseudocode?

Company: Infosys, Accenture

```
class Demo {
    { print "Instance"; }
    Demo() { print "Constructor"; }
}
Demo d = new Demo();
// Demo d1 = new Demo();
```

A) Instance Constructor
B) Constructor Instance
C) Error
D) Instance

Explanation: Constructor Chaining in Inheritance

Instance initializer blocks (code within curly braces directly in the class, not inside a method or constructor) are executed every time an object of the class is created, and they run **before** the constructor. Thus, "Instance" will be printed first, followed by "Constructor".

Correct Answer: A) Instance Constructor

Q23. What will be the output of the following pseudocode?

Company: Infosys, Wipro

```
class Test {
    Test() { this(5); print "No arg "; }
    Test(int x) { print "Int arg "; }
}
Test t = new Test();
```

A) No arg Int arg
B) Int arg No arg
C) Error
D) No output

Explanation: Constructor Overloading with ‘this()’

The **this(5)** call within the no-argument constructor invokes the integer-argument constructor. The integer-argument constructor executes first, printing "Int arg ". After its completion, the execution returns to the no-argument constructor, which then prints "No arg ". This is a common pattern for constructor chaining within the same class.

Correct Answer: B) Int arg No arg

Q24. What will be the output of the following pseudocode?

Company: Capgemini, Wipro

```
class Test {
    static int count = 0;
    Test() { count++; }
}
Test t1 = new Test();
Test t2 = new Test();
print Test.count;
```

A) 1
B) 2
C) 0
D) Error

Explanation: Static Variable and Object Instantiation

The **count** variable is **static**, meaning it belongs to the class itself, not to any specific instance. Each time a **Test** object is created (via its constructor), the **count** is incremented. Since two objects (**t1** and **t2**) are created, the **count** will be incremented twice, resulting in a final value of 2.

Correct Answer: B) 2

Q25. What will be the output of the following pseudocode?

Company: Infosys , Accenture

```
class A {
    protected void show() { print "A"; }
}
class B extends A {
    public void show() { print "B"; }
}
B b = new B();
b.show();
```

A) A
B) B
C) Error
D) No output

Explanation: Method Overriding and Access Modifiers

When overriding a method, the access modifier in the subclass must be either the same or more lenient (less restrictive) than in the superclass. Here, **protected** is less restrictive than **public**, so this is a valid override. The **show()** method in class **B** successfully overrides the one in class **A**. When **b.show()** is called, the overridden method in **B** is executed.

Correct Answer: B) B

Q26. What will be the output of the following pseudocode?

Company: Infosys , Capgemini

```
class Outer {
    int x = 10;
    class Inner {
        void show() { print x; }
    }
}
Outer.Inner obj = new Outer().new Inner();
obj.show();
```

A) 0
B) Error
C) 10
D) Null

Explanation: Accessing Outer Class Members in Inner Class

Inner classes in Java (and similar concepts in other languages) have implicit access to the members of their enclosing outer class, including private members. When an instance of **Inner** is created through an instance of **Outer**, the **show()** method can directly access the **x** variable of the **Outer** instance.

Correct Answer: C) 10

Q27. What will be the output of the following pseudocode?

Company: TCS NQT , Accenture

```
interface A { void show(); }
A obj = new A() {
    public void show() { print "Anonymous"; }
};
obj.show();
```

A) A
B) Anonymous
C) Error
D) No output

Explanation: Anonymous Inner Class

Explanation: This code uses an anonymous inner class to implement the **A** interface on the fly. The **show()** method within this anonymous class is defined to print "Anonymous". When **obj.show()** is called, it invokes this specific implementation.

Correct Answer: B) Anonymous

Q28. What will be the output of the following pseudocode?

Company: TCS NQT , Accenture

```
interface A { void show(); }
A obj = new A() {
    public void show() { print "Anonymous"; }
};
obj.show();
```

A) A
B) Anonymous
C) Error
D) No output

Explanation: Anonymous Inner Class

Explanation: This code uses an anonymous inner class to implement the **A** interface on the fly. The **show()** method within this anonymous class is defined to print "Anonymous". When **obj.show()** is called, it invokes this specific implementation.

Correct Answer: B) Anonymous

Q29. What will be the output of the following pseudocode?

Company: Infosys, Accenture

```
class Test implements Cloneable {
    int x = 5;
    public Object clone() { return super.clone(); }
}
Test t1 = new Test();
Test t2 = (Test)t1.clone();
print t2.x;
```

A) 5
B) 0
C) Error
D) Null

Explanation: Object Cloning

The **clone()** method creates a shallow copy of the object. This means that primitive fields like **x** will have their values copied directly to the new object. So, **t2.x** will be 5, the same value as **t1.x** at the time of cloning.

Correct Answer: A) 5

Q30. What will be the output of the following pseudocode?

Company: Infosys, Wipro

```
class A {
    void show() throws IOException { print "A"; }
}
class B extends A {
    void show() throws FileNotFoundException { print "B"; }
}
B b = new B();
b.show();
```

A) A
B) B
C) Error
D) No output

Explanation: Exception Handling in Method Overriding

When overriding methods that throw exceptions, the child class can only throw:

- The same exception as the parent
- A subclass of the parent's exception
- No exception at all

Since FileNotFoundException is a subclass of IOException, this code compiles and runs successfully.

Correct Answer: B) B

Q31. What will be the output of the following pseudocode?

Company: Capgemini, Wipro

```
class Test {
    private Test() { }
    static Test getInstance() { return new Test(); }
}
Test t = Test.getInstance();
```

A) Error
B) Success
C) No output
D) Null

Explanation: Private Constructors and Factory Method Pattern

This code demonstrates the Factory Method pattern, where object creation is handled through a static method rather than direct constructor calls.

- Private constructor prevents direct instantiation
- Static method provides controlled object creation
- Code compiles and executes successfully

Correct Answer: B) Success

Q32. What will be the output of the following pseudocode?

Company: Infosys, Accenture

```
class Test {
    void show(int x) { print "int"; }
    void show(double x) { print "double"; }
}
Test t = new Test();
t.show('A');
```

A) int
B) double
C) Error
D) No output

Explanation: Method Overloading and Type Promotion

The character 'A' is automatically promoted to int (ASCII value 65) and the show(int) method is called, resulting in "int" being printed.

Type Promotion Rules

When a method is called with a char parameter but no char overload exists:

- Java promotes char to int (if an int version exists)
- If no int version, char is promoted to wider types (long, float, double)
- Promotion follows the path of least widening

Correct Answer: A) int

Q33. What will be the output of the following pseudocode?

Company: Infosys, Capgemini

```
class A {
    A get() { return this; }
}
class B extends A {
    B get() { return this; }
}
B b = new B();
print b.get() instanceof B;
```

A) true
B) false
C) Error
D) Null

Explanation: Covariant Return Type

Since Java 5, method overriding allows a subclass method to return a more specific type (subclass) than the overridden method in the parent class.

In this example:

- Class A's get() returns type A
- Class B's get() returns type B (more specific)
- b.get() returns a B object
- instanceof checks if the object is of type B

Correct Answer: A) true

Q34. What will be the output of the following pseudocode?

Company: TCS NQT , Accenture

```
class Test {
    static void show() { print "Static"; }
}
Test t = null;
t.show();
```

A) Null
B) Error
C) Static
D) No output

Explanation: Static Method Invocation with Null Reference

Static methods belong to the class, not to any instance. Key characteristics:

- Can be called using class name: Test.show()
- Can also be called using an instance reference
- NullPointerException is NOT thrown when called using a null reference
- The compiler resolves the call to the class, not the reference

Correct Answer: C) Static

Q35. What will be the output of the following pseudocode?

Company: TCS NQT

```
class Parent {
    int x = 5;
}
class Child extends Parent {
    int x = 10;
    void printX() { print x + " " + super.x; }
}
Child c = new Child();
c.printX();
```

A) 10 5
B) 5 10
C) 10 10
D) 5 5

Explanation: Variable Shadowing and the Super Keyword

The Child class's x (10) is printed first, followed by the Parent class's x (5) accessed via super.

Variable Shadowing

When a subclass declares a variable with the same name as in the parent class, the subclass variable shadows (hides) the parent variable.

Super Keyword

The super keyword provides access to the parent class's shadowed variables and overridden methods, allowing explicit reference to parent class members.

Correct Answer: A) 10 5

Q36. What will be the output of the following pseudocode?

Company: Wipro

```
abstract class A {
    void show() { print "A"; }
private void display() {   print ”display”; }
protected void M1() { print ”Methdod 1”;}
}
class B extends A { }
B b = new B();
b.show();
```

A) A
B) B
C) Error
D) No output

Explanation: Method Inheritance

When a class extends another class, it inherits all non-private methods from the parent class.

Class B extends abstract class A but doesn't override the show() method.

When b.show() is called, it uses the inherited implementation from class A.

Correct Answer: A) A

Q37. What will be the output of the following pseudocode?

Company: Accenture

```
class A {
    A() { show(); }
    void show() { print "A"; }
}
class B extends A {
    int x = 5;
    void show() { print x; }
}
B b = new B();
```

A) 0
B) 5
C) Error
D) A

Explanation: Constructor Chaining and Method Overriding

Code Execution Flow:

Create instance of B, which first calls A's constructor

A's constructor calls show()

Since B overrides show(), B's version is called

At this point, B's instance variables aren't initialized yet

This question tests understanding of Java's initialization order. When a subclass constructor runs, the parent constructor executes first. If the parent constructor calls an overridden method, the child's version runs - but instance variables in the child class haven't been initialized yet, so x is 0 (default value).

Correct Answer: A) 0

Q38. What will be the output of the following pseudocode?

```
class A {
    static void show() { print "A"; }
}
class B extends A {
    void show() { print "B"; }
}
B b = new B();
b.show();
```

A) A
B) B
C) Error
D) AB

Explanation: Static Method Inheritance

This code has a compile-time error. A static method in a parent class cannot be overridden with a non-static method in a child class. Static methods belong to the class, not to instances, and method overriding applies to instance methods.

Correct Answer: C) Error

Q39. What will be the output of the following pseudocode?

Company: Capgemini

```
class Parent {
    static void show() { print "Parent"; }
}
class Child extends Parent {
    static void show() { print "Child"; }
}
Parent p = new Child();
p.show();
```

A) Parent
B) Child
C) Error
D) No output

Explanation: Static Method Binding

Since p is declared as type Parent, the compiler binds the call to Parent.show(), regardless of the actual object being a Child instance. This is different from instance method overriding, which uses dynamic binding.

Key Concept

Static methods are bound at compile time (early binding), not at runtime. The declared type of the reference variable determines which static method is called, not the actual object type.

Correct Answer: A) Parent

Q40. What will be the output of the following pseudocode?

```
class Test {
    void show(final int x) {
        // x = 10;
        print x;
    }
}
Test t = new Test();
t.show(5);
```

A) Error

B) Null

C) 5

D) 10

Explanation: Final Parameters in Method

In this code, the parameter x is declared as final, and there's a commented line that would attempt to modify it. Since this line is commented out, the code will compile and run without issues.

Understanding final Parameters

When a parameter is declared as final in a method, it means its value cannot be changed within the method. Any attempt to reassign a value to a final parameter would result in a compilation error.

Correct Answer: C) 5

Q41. What will be the output of the following pseudocode?

```
class A {
    A() { print "A"; }
    A(int x) { print x; }
}
class B extends A {
    B() { super(5); print "B"; }
}
B b = new B();
```

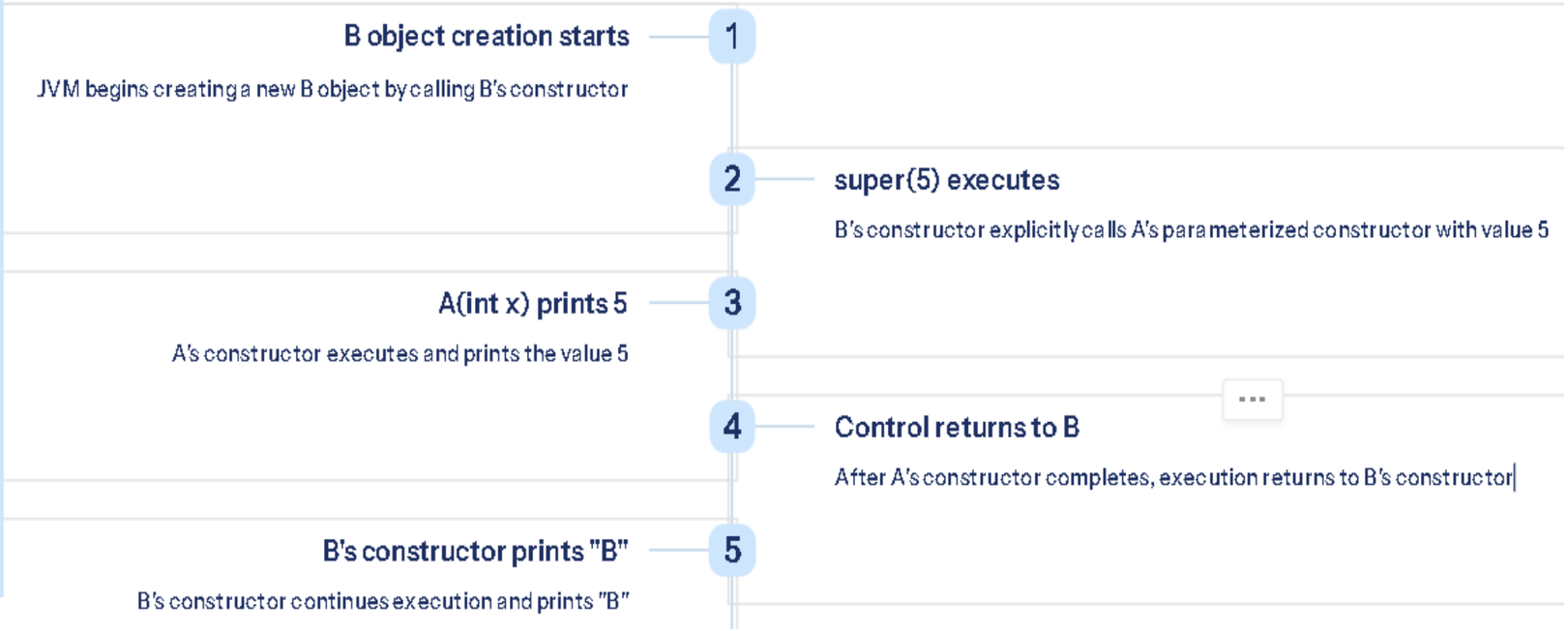
A) 5 B

B) A B

C) B 5

D) Error

Explanation: Constructor Chaining With Super()



Correct Answer: A) 5 B

Q42. What will be the output of the following pseudocode?

```
class Outer {
    int x = 10;
    void test() {
        Object obj = new Object() {
            int x = 20;
            void printX() { print x; }
        };
        // obj.printX();
    }
}
new Outer().test();
```

A) 10
B) 20
C) Error
D) No output

Explanation: Anonymous Inner Class

This code creates an anonymous inner class that extends Object and adds a field x and a method printX(). However, since Object doesn't have a printX() method, we would need to cast obj or use reflection to access this method.

The key point here is that the call to printX() is commented out. Without this call, the method is never invoked, so nothing is printed to the console.
Even if the line were uncommented, it would cause a compile-time error because Object doesn't have a printX() method, and the reference type of obj is Object, not the anonymous class type.

Correct Answer: D) No output

Q43. What will be the output of the following pseudocode?

Company: Capgemini, Cognizant

```
class Test {
    void show(String s) { print "String"; }
    void show(Object o) { print "Object"; }
}
Test t = new Test();
t.show(null);
```

A) String

B) Object

C) Error

D) Null

Explanation: Method Overloading Resolution For Null

When calling an overloaded method with null, Java selects the most specific matching method. In this case, String is more specific than Object because String is a subclass of Object.

null Value

null can be assigned to any reference type in Java, but the compiler chooses the most specific type when resolving method calls with null arguments.

Correct Answer: A) String